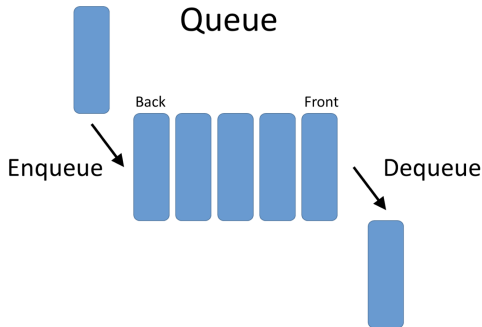


Introdução à Computação tipos abstratos, filas

Alexandre Rademaker

filas I

Outro tipo abstrato de dados é a **fila**, onde valores são colocados em uma extremidade e retirados da extremidade oposta.



filas II

Fila um tipo abstrato de dados do tipo “First-In, First Out” (FIFO), ou seja “o primeiro que entra é o primeiro que sai”.

Filas podem ser implementadas com listas encadeadas, arrays, arrays circulares.

filas III

Filas são usadas em várias áreas da computação onde o acesso a algum recurso é controlado e requisições devem aguardar para serem atendidas: CPU, fila de impressão, roteadores etc.

Na navegação em grafos, **buscas em profundidade** são implementadas com **pilhas** e **buscas em largura** com **filas**.

filas IV

As duas principais operações com filas são:

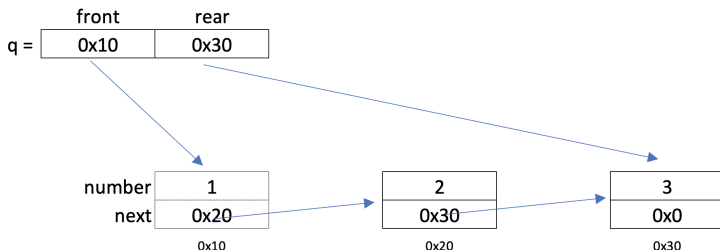
dequeue retira um elemento da frente da fila. Na implementação com arrays pode ser $O(n)$ mas com listas encadeadas é $O(1)$.

enqueue insere um elemento no final da fila. Complexidade $O(1)$.

Outras operações possíveis (e auxiliares): imprimir a fila, inspecionar o primeiro elemento, inspecionar o último elemento, determinar se a fila está vazia, tamanho da fila, etc.

filas V

Em C, vamos implementar uma struct contendo dois ponteiros, um para a **frente** e outro para o **final** da fila.



filas em C I

Além da estrutura já criada para listas encadeadas, uma estrutura adicional para conter os o ponteiro para o início da fila (front) e final da fila (rear).

queue/queue-list.c

```
1 typedef struct node {
2     int number;
3     struct node *next;
4 } t_node;
5
6 typedef struct queue {
7     t_node *front;
8     t_node *rear;
9 } t_queue;
```

filas em C II

queue/queue-list.c

```
1 void enqueue(t_queue* q, int k) {
2     t_node* temp = create_node(k);
3
4     // if empty
5     if (q->rear == NULL) {
6         q->front = q->rear = temp;
7         return;
8     }
9
10    q->rear->next = temp;
11    q->rear = temp;
12 }
```


filas em C III

queue/queue-list.c

```
1 void dequeue(t_queue* q) {  
2     // if empty  
3     if (q->front == NULL) return;  
4  
5     // save previous front and move  
6     t_node* temp = q->front;  
7     q->front = q->front->next;  
8  
9     // if front becomes NULL  
10    if (q->front == NULL)  
11        q->rear = NULL;  
12  
13    free(temp);  
14 }
```

filas em C IV

Using it

```
1 t_queue* create_queue() {
2     t_queue *q = malloc(sizeof(t_queue));
3     q->front = q->rear = NULL;
4     return q;
5 }
6
7 int main(int argc, char *argv[]) {
8     t_queue *q = create_queue();
9     enqueue(q, 1);
10    enqueue(q, 2);
11    dequeue(q);
12    enqueue(q, 3);
13    enqueue(q, 4);
14    dequeue(q);
15 }
```